

One Hot Encoding

Module 27 Study Notes • Feature Engineering

1. Introduction: Handling Nominal Data

In the previous module, we learned how to use Ordinal Encoding for categorical data that possesses an inherent ranking (e.g., "Poor" < "Average" < "Good"). However, a massive portion of real-world categorical data is **Nominal**, meaning it has zero intrinsic mathematical order. Examples include Car Brands (Maruti, Hyundai, Ford), Colors (Red, Blue, Green), or States (Delhi, Maharashtra, Karnataka).

If we apply Ordinal Encoding to Nominal data and map 'Red' to 1, 'Blue' to 2, and 'Green' to 3, the machine learning algorithm will falsely assume that Green is mathematically "greater than" Red. To prevent this dangerous assumption, we use a technique called **One Hot Encoding (OHE)**.

2. How One Hot Encoding Works

Instead of mapping multiple categories to a single numerical column, One Hot Encoding takes a single column and splits it into multiple new **binary columns**—one for each unique category.

For example, if the column "Color" has three categories (Yellow, Blue, Red), OHE creates three new columns: **Color_Yellow**, **Color_Blue**, and **Color_Red**. Each row will have a **1** in the column that matches its category, and a **0** in all others. You are effectively transforming the string into a binary vector.

3. The Dummy Variable Trap (Multicollinearity)

When you create these new binary columns (often called "dummy variables"), you introduce a mathematical problem known as **Multicollinearity**.

The Trap: If you have 3 color categories, you only actually need 2 columns to perfectly represent the data. If a row is NOT Yellow (0) and NOT Blue (0), it mathematically **MUST** be Red. Including the 'Red' column is redundant. In linear models (like Linear Regression or Logistic Regression), having perfectly predictable columns causes the algorithm to crash or behave erratically.

The Solution: You must drop one column from the generated set. If a category had 'N' unique values, you should only keep **(N - 1)** columns.

4. Implementation Workflows

A. Using Pandas (Great for EDA, Bad for Production)

Pandas provides a very fast, built-in function to perform OHE called `get_dummies()`. You can easily solve the Dummy Variable Trap by using the `drop_first=True` parameter.

```
import pandas as pd

# This will automatically replace the categorical columns with N-1 binary columns
df_encoded = pd.get_dummies(df, columns=['Fuel_Type', 'Owner'], drop_first=True)
```

Warning: While `get_dummies` is excellent for quick exploration, it does not "remember" the columns it created. If your Test set is missing a category that was present in your Train set, the shapes will mismatch, and your model will crash. Therefore, you should use Scikit-Learn for production.

B. Using Scikit-Learn (The Production Standard)

Scikit-Learn's `OneHotEncoder` class is designed to be used in ML pipelines. It remembers the categories it learned during the `fit()` step and guarantees that the Test set will have the exact same shape.

```
from sklearn.preprocessing import OneHotEncoder
from sklearn.model_selection import train_test_split

# 1. Train-Test Split
X_train, X_test, y_train, y_test = train_test_split(df.iloc[:, 0:4], df.iloc[:, -1], test_size=0.2)

# 2. Initialize Encoder (drop='first' solves the Dummy Variable Trap)
# sparse=False returns a NumPy array instead of a Sparse Matrix
ohe = OneHotEncoder(drop='first', sparse=False)

# 3. Fit and Transform on specific columns
# Note: Manually slicing columns like this is tedious. Module 28 will introduce
# ColumnTransformer to solve this!
X_train_ohe = ohe.fit_transform(X_train[['Fuel_Type', 'Owner']])
```

```
X_test_ohe = ohe.transform(X_test[['Fuel_Type', 'Owner']])
```

5. Handling High Cardinality (The "Other" Category)

A major drawback of OHE is the **Curse of Dimensionality**. If your column is "Car Brand" and there are 40 different brands, OHE will add 39 new columns to your dataset, drastically slowing down processing and potentially causing overfitting.

The Strategy: Identify a threshold frequency. For instance, keep the top 10 most frequent car brands as their own columns, and lump all the remaining 30 rare brands into a single new category named **"Other"** or **"Uncommon"**. This drastically reduces the column explosion while preserving the most important data.